Chapter 17 — Spring AI and What's New in Boot 4 / Framework 7

Two halves. First, **Spring AI**: the Spring project for calling large language models (LLMs) through one API across providers (OpenAI, Anthropic, Google, Amazon Bedrock, Ollama, ...), plus what a real AI feature needs beyond "send text, get text": prompt templates (the **prompt** is the text sent to the model), conversation memory, answers as Java objects, letting the model call your code (tools), searching your documents (RAG, retrieval-augmented generation, §17.9) and the Model Context Protocol (MCP: a standard way to offer tools to AI apps, §17.10). Second, a **migration map**: Boot 3.x → 4.x, Framework 6 → 7, Security 6 → 7, each change pointing to the chapter that teaches it.

How LLMs work is **not** re-taught. Tokens (the word pieces a model reads, writes and bills by), context windows (max text per call), sampling (how each next token is picked), prompting, embeddings (number lists capturing a text's meaning, §17.9), RAG design and agents (a model repeatedly choosing the next tool) are in the AI book: LLMs, prompt engineering, building LLM apps, embeddings, vector databases and RAG, evaluation, agents. Here: the **Spring way**.

The code. `code/spring-ai-offline/` is a Spring Boot 4.1.1 project (Framework 7.0, Java 25) on **Spring AI 2.0.1**, needing **no API key** (the provider's authentication/billing secret) **and no network**: the `ChatModel` bean is a fake returning canned answers (`FakeChatModel`), the `EmbeddingModel` a word-hashing fake (`FakeEmbeddingModel`). Everything *above* the model — `ChatClient`, advisors, memory, tool calling, structured output, the vector store — is real Spring AI. Output is real `mvn -q -B -ntp verify` output (14 tests passing). Snippets needing a real provider are marked **(needs an API key — not run)**.

17.1 What Spring AI is

What it is

Spring AI does for AI providers what Spring Data does for databases: **portable interfaces** (`ChatModel`, `EmbeddingModel`, `VectorStore`, ...), **Boot starters** auto-configuring a provider from properties, and a **fluent client** (`ChatClient`). As `RestClient` (Chapter 6) hides the HTTP library, `ChatClient` hides the provider: switching OpenAI → Anthropic is mostly a starter and property change.

Why it matters

Calling an LLM's HTTP API directly is fine for a demo. Production also needs prompt templates outside the code, conversation memory (the model remembers nothing between calls), typed Java answers, tool calling, retrieval of your documents (RAG), and logging/metrics/tracing/retries plus tests that don't call a paid API. Spring AI provides each as a Spring building block — beans, properties, Micrometer observations (Chapter 15), Boot tests (Chapter 12).

Versions (verified September 2026)

Spring AI	Released	Works with	Notes
1.0	May 2025	Boot 3.4 / 3.5	first GA (general availability)
1.1	Nov 2025	Boot 3.4 / 3.5	MCP annotations, more providers
2.0	**Jun 2026**	**Boot 4.0 / 4.1, Framework 7**	Jackson 3, JSpecify null-safety (annotations marking what may be `null`), tool loop moved into advisors, immutable options, MCP Java SDK 2.0. **Current: 2.0.1** (used here)

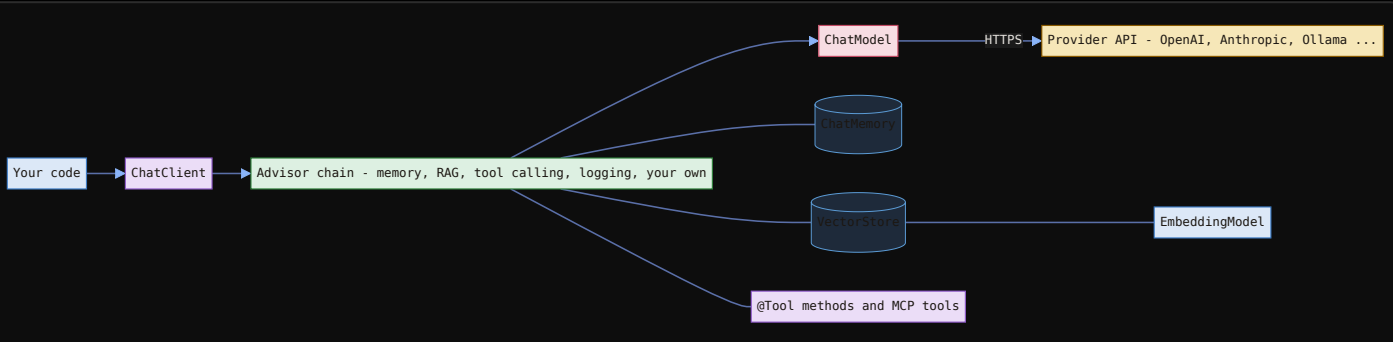
Spring AI 2.0 **requires** Boot 4; a Boot 3.5 app stays on 1.1.x until it migrates. 2.0.0's starters were built against Boot 4.1 dependency versions (spring-ai issue #6465), so Boot 4.1.x is smoothest.

Boot doesn't manage Spring AI's version, so import the **Spring AI BOM** (bill of materials), which keeps all `spring-ai-*` artifacts aligned:

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.springframework.ai</groupId>
      <artifactId>spring-ai-bom</artifactId>
      <version>2.0.1</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>
```

The building blocks

Building block	What it does	Real implementations
`ChatModel`	low-level, provider-level: sends a `Prompt` (messages + options), returns a `ChatResponse`	`OpenAiChatModel`, `AnthropicChatModel`, `OllamaChatModel`, ... (our `FakeChatModel`)
`ChatClient`	the fluent, everyday API on top of a `ChatModel`, with advisors	one implementation, built from a `ChatClient.Builder`
Advisor	an interceptor around every `ChatClient` call (like a servlet filter)	logging, memory, RAG, tool calling, safeguard, validation
`ChatMemory`	stores past messages per conversation	`MessageWindowChatMemory` over in-memory / JDBC / other repositories
Structured output	converts the text answer into a Java type	`BeanOutputConverter`, `.entity(...)`
`ToolCallback` / `@Tool`	a Java method the model may ask to run	method tools, function tools, MCP tools
`EmbeddingModel`	turns text into a vector (fixed-length list of numbers)	OpenAI, Ollama, ONNX transformers, ... (our `FakeEmbeddingModel`)
`VectorStore`	stores document vectors and finds similar ones	`SimpleVectorStore` (in memory), PgVector (PostgreSQL vector extension), Redis, Elasticsearch, Qdrant, ...
MCP	standard protocol to expose / consume tools across processes	MCP server and client starters



17.2 Setting up a project

With a real provider (needs an API key — not run)

1. Add the BOM (above) and **one model starter**:

```
<!-- OpenAI -->
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-starter-model-openai</artifactId>
</dependency>
<!-- or Anthropic (Claude) -->
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-starter-model-anthropic</artifactId>
</dependency>
```

2. Configure it — key in an environment variable or secret store, **never** in Git (Chapter 3 explains `${...}`):

```
# OpenAI (needs an API key — not run)
spring.ai.openai.api-key=${OPENAI_API_KEY}
spring.ai.openai.chat.model=gpt-5-mini
spring.ai.openai.chat.temperature=0.2
spring.ai.openai.embedding.model=text-embedding-3-small
```

```
# Anthropic (needs an API key — not run)
spring.ai.anthropic.api-key=${ANTHROPIC_API_KEY}
spring.ai.anthropic.chat.model=claude-sonnet-5-5
# a larger model for harder tasks: claude-opus-5-5
spring.ai.anthropic.chat.max-tokens=4096
```

3. Inject `ChatClient.Builder` and use it (`temperature` controls randomness — lower is more predictable; `max-tokens` caps answer length). The rest of the chapter works unchanged with either provider.

Spring AI 2.0 configuration facts (from the 2.0 upgrade notes):

Change in 2.0	Before (1.x)	Now (2.0)
option properties are **flat**	`spring.ai.openai.chat.options.model=...`	`spring.ai.openai.chat.model=...`
Anthropic module	Spring's own `AnthropicApi` client	the official `anthropic-java` SDK; default `max-tokens` raised from 500 to 4096
OpenAI modules	three variants (`openai`, `openai-sdk`, `azure-openai`)	one `spring-ai-openai` (Azure/Microsoft Foundry through its properties)
options objects	mutable, `copy()`	immutable, `mutate()`; `ChatClient.options(...)` takes a **builder**
several model starters on the classpath	—	pick one with `spring.ai.model.chat=openai` / `anthropic` / ...

Default models change (2.0's OpenAI default is `gpt-5-mini`, Anthropic's `claude-haiku-4-5`). Always set the model explicitly so an upgrade doesn't silently change cost and behaviour.

Offline, as in this chapter

Our `pom.xml` has the BOM plus these artifacts and **no model starter**:

Artifact	Why
`spring-boot-starter`	a plain (non-web) Boot app
`spring-ai-autoconfigure-model-chat-client`	auto-configures a `ChatClient.Builder` bean from whatever `ChatModel` bean exists
`spring-ai-autoconfigure-model-tool`	the tool-calling manager and its properties (an *optional* dependency of the previous one, so add it yourself — else Boot logs `TypeNotPresentException ... ToolCallingAutoConfiguration`)
`spring-ai-autoconfigure-model-chat-memory`	a `ChatMemory` bean (window of the last 20 messages, stored in memory)
`spring-ai-vector-store`, `spring-ai-vector-store-advisor`	`SimpleVectorStore` and `QuestionAnswerAdvisor`

A model starter would pull these in. We declare our own `ChatModel` and `EmbeddingModel` beans (`AiConfig`):

```
@Configuration
public class AiConfig {

    /** With a real starter on the classpath, Boot would create OpenAiChatModel / AnthropicChatModel here. */
    @Bean
    FakeChatModel chatModel() { return new FakeChatModel(); }

    @Bean
    EmbeddingModel embeddingModel() { return new FakeEmbeddingModel(); }
    // ... vectorStore and supportClient beans, shown later
}
```

Why a fake model?

A **fake** (a test double with simple working logic, Chapter 12) tests everything around the model without cost, network, rate limits or randomness. Real LLM output is **non-deterministic** (the same prompt gives different words), so tests asserting exact LLM text are flaky. The usual strategy:

What you test	How
your prompts, advisors, tools, parsing, error handling	fake or stub `ChatModel` — fast, deterministic, runs in CI
"is the answer good?"	a small **evaluation** set run against the real model, scored by rules or by another model (Spring AI has `RelevancyEvaluator` and `FactCheckingEvaluator`); see AI 8.5
the provider integration	a few tagged integration tests with a real key, run on demand

---

17.3 `ChatModel` and `ChatClient`

`ChatModel`: the provider contract

`ChatModel` has one essential method, `ChatResponse call(Prompt prompt)`. A `Prompt` is a list of **messages** plus **options** (model, temperature, tools, ...); a `ChatResponse` holds `Generation`s, each with an `AssistantMessage`.

Message type	Class	Who writes it
system	`SystemMessage`	you: the rules and persona ("You are a support assistant...")
user	`UserMessage`	the end user's question (often via a template)
assistant	`AssistantMessage`	the model's reply; may contain **tool calls** instead of text
tool	`ToolResponseMessage`	your application: the result of a tool the model asked for

Our fake implements exactly this interface (`FakeChatModel`), recording each prompt so tests can show what reached "the model", then applying a few rules:

```
public class FakeChatModel implements ChatModel {

    private final List<Prompt> received = new CopyOnWriteArrayList<>();

    @Override
    public ChatResponse call(Prompt prompt) {
        received.add(prompt);                                   // remember what the model was sent
        return new ChatResponse(List.of(new Generation(reply(prompt))));
    }

    /** Tool-calling-capable default options, like a real provider's options class. */
    @Override
    public ChatOptions getOptions() {
        return ToolCallingChatOptions.builder().build();
    }

    private AssistantMessage reply(Prompt prompt) {
        Message last = prompt.getInstructions().getLast();
        if (last instanceof ToolResponseMessage toolResult) { /* Rule 1: explain the tool result */ }
        String user = prompt.getUserMessage().getText();
        // Rule 2: tools offered + an order id in the question -> return an AssistantMessage with a ToolCall
        // Rule 3: "JSON Schema" in the prompt -> return JSON in a ```json fence
        // Rule 4: RAG context present -> answer from its first line
        // Rule 5: "what is my name" -> look for "my name is X" in earlier messages
        return new AssistantMessage("(fake model) You asked: " + user);
    }
    // ...
}
```

`getOptions()` matters: `ChatClient` copies tool definitions into the prompt's options only when the model's options type supports tools (`ToolCallingChatOptions`), as every real provider's does.

`ChatClient`: the everyday API

Boot auto-configures a **prototype-style** `ChatClient.Builder` bean (a new builder per injection point), wired to the `ChatModel` bean and observability. Customise it and `build()` a thread-safe `ChatClient` meant for reuse (make it a bean).

```
@SpringBootTest
class ChatClientTests {

    @Autowired ChatClient.Builder builder;    // auto-configured by Spring AI from the ChatModel bean
    @Autowired FakeChatModel model;

    @Test
    void simplestCall() {
        ChatClient chat = builder.build();
        String answer = chat.prompt()          // start a request
                .user("What is Spring AI?")    // the user message
                .call()                        // blocking call (use .stream() for a Flux)
                .content();                    // just the text of the reply
        System.out.println("answer: " + answer);
        assertThat(answer).isEqualTo("(fake model) You asked: What is Spring AI?");
    }
}
```

Output:

```
answer: (fake model) You asked: What is Spring AI?
```

What `call()` can return:

Method	Returns	Use when
`.content()`	`String`	you only need the text
`.entity(Type.class)`	your Java type	structured output (§17.7)
`.chatResponse()`	`ChatResponse`	metadata: token usage, finish reason (why the model stopped: done, length limit), model id
`.chatClientResponse()`	response + advisor context	you need what advisors stored, e.g. the retrieved documents
`.stream().content()`	`Flux<String>`	show tokens as they arrive (chat UIs, SSE — Chapter 13)

Common mistakes

- A new `ChatClient` per request instead of one bean per use case.
- Calling `ChatModel` directly everywhere, losing advisors, memory and observability — use `ChatClient` unless writing infrastructure.
- Assuming the model remembers the previous call (§17.6).

17.4 Prompts and prompt templates

What it is

A **prompt template** is text with placeholders filled at run time — like SQL bind parameters. `PromptTemplate` uses `{name}` placeholders (StringTemplate engine; change delimiters with `StTemplateRenderer` if the text contains braces, e.g. JSON examples).

How it works

1. Write the template inline or in a file such as `prompts/explain.st`: `Explain {topic} to a {audience} in at most {words} words.`
2. Supply values with `render(Map)` / `create(Map)`, or `.param(...)` on `ChatClient`.
3. A **missing** value is an error (`IllegalStateException` listing the names), not an empty string — typos fail fast.

```
PromptTemplate template = new PromptTemplate(
        "Explain {topic} to a {audience} in at most {words} words.");
String text = template.render(Map.of("topic", "dependency injection", "audience", "beginner", "words", 50));

PromptTemplate fromFile = new PromptTemplate(new ClassPathResource("prompts/explain.st"));
Prompt prompt = fromFile.create(Map.of("topic", "virtual threads", "audience", "Java developer", "words", 80));

new PromptTemplate("Hello {name}, your plan is {plan}.").render(Map.of("name", "Ana"));   // throws
```

Output (`PromptTemplateTests`):

```
rendered: Explain dependency injection to a beginner in at most 50 words.
prompt messages: 1, user text: Explain virtual threads to a Java developer in at most 80 words.
missing variable -> IllegalStateException: Not all variables were replaced in the template. Missing variable names
are: [plan].
```

With `ChatClient`, templates and a system message look like this:

```java
@Value("classpath:prompts/explain.st") Resource explainTemplate;

ChatClient chat = builder.defaultSystem("You are a concise Java tutor.").build();
chat.prompt()
    .user(u -> u.text(explainTemplate)
                .param("topic", "records")
                .param("audience", "beginner")
                .param("words", 40))
    .call()
    .content();
```

Output — the messages the model actually received:

```
messages the model received:
  SYSTEM: You are a concise Java tutor.
  USER: Explain records to a beginner in at most 40 words.
```

`defaultSystem(...)`, `defaultUser(...)`, `defaultAdvisors(...)`, `defaultTools(...)` and `defaultOptions(...)` apply to every request of that client; per-request `.system(...)`, `.user(...)`, `.advisors(...)`, `.tools(...)`, `.options(...)` add to or override them.

Common mistakes

- Concatenating user input into the **system** message — makes **prompt injection** (text crafted to override your instructions) easier (AI 9.4). User text belongs in the user message.
- Templates buried in Java strings — put long prompts in `src/main/resources/prompts/*.st`, loaded as `Resource`, reviewed and versioned like code.
- JSON examples with `{...}` in a template without changing delimiters — the braces become placeholders.

17.5 Advisors

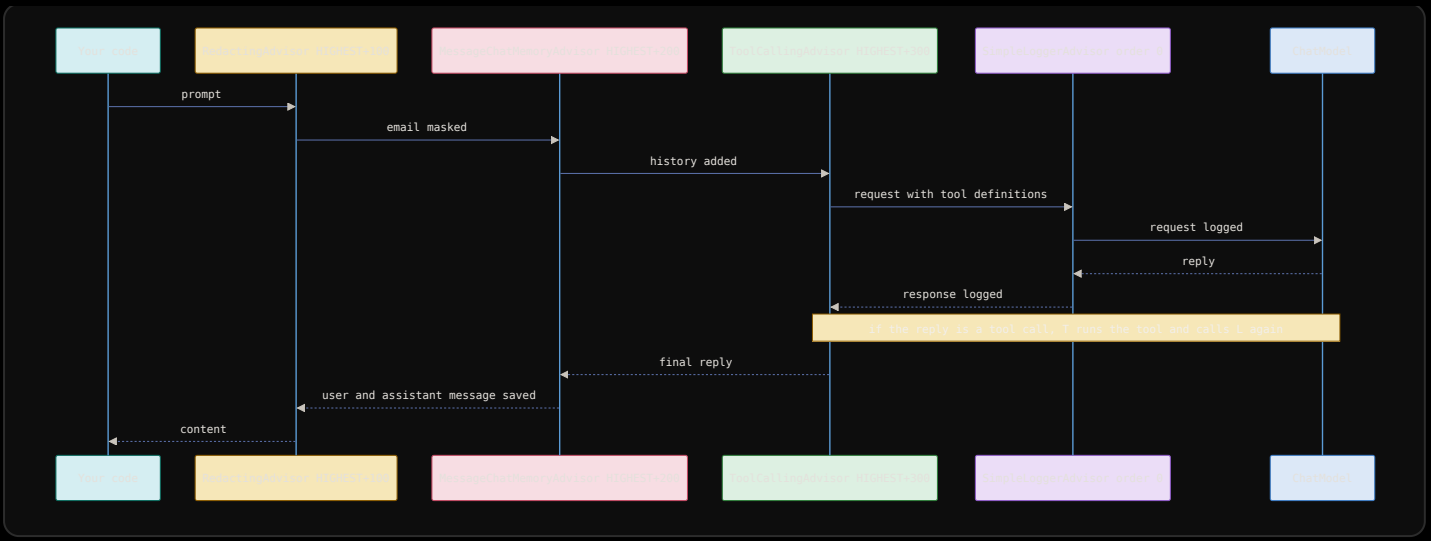
What it is

An **advisor** intercepts every `ChatClient` call on the way **to** the model and **back**. It can change the prompt (add memory or retrieved documents, redact data), change the response, or short-circuit the call (answer without the model). Advisors are to `ChatClient` what servlet filters are to Spring MVC (5.11) and `@Around` advice is to AOP (Chapter 4).

How it works

1. `ChatClient` builds a `ChatClientRequest` (the `Prompt` + a shared **context** map).
2. Advisors are sorted by `getOrder()` — **lower runs first** on the way in and **last** on the way out.
3. Each advisor calls `chain.nextCall(request)` to pass control on. The last link calls the `ChatModel`.
4. The response travels back through the advisors in reverse order.

`HIGHEST` is `Ordered.HIGHEST_PRECEDENCE` (the smallest `int`). Real Spring AI 2.0 defaults: memory advisors `HIGHEST_PRECEDENCE + 200`, the auto-registered `ToolCallingAdvisor` `HIGHEST_PRECEDENCE + 300`, `SimpleLoggerAdvisor` `0` — so the logger sits *inside* the tool loop (logging every model round trip) and memory outside it (saving only the final exchange).



Built-in advisors (Spring AI 2.0)

Advisor	Purpose	Section
`SimpleLoggerAdvisor`	logs request and response at DEBUG	below
`MessageChatMemoryAdvisor`	adds past messages of the conversation to the prompt, saves the new exchange	§17.6
`VectorStoreChatMemoryAdvisor`	long-term memory: retrieves *relevant* past messages from a vector store	§17.6
`QuestionAnswerAdvisor`	simple RAG: similarity search (closest stored texts in meaning) + context pasted into the user message	§17.9
`RetrievalAugmentationAdvisor`	modular RAG: query rewriting, expansion, retrieval, joining, augmentation	§17.9
`ToolCallingAdvisor`	runs the tool-call loop; **auto-registered** by `ChatClient` (new in 2.0; replaced the loop inside each `ChatModel`)	§17.8
`SafeGuardAdvisor`	blocks requests containing configured sensitive words	—
`StructuredOutputValidationAdvisor`	validates JSON output against the schema and asks the model to retry if it does not match (new in 2.0)	§17.7

A custom advisor: redacting e-mail addresses

Many companies must not send personal data to an external provider; an advisor enforces that once, for every call (`RedactingAdvisor`):

```
public class RedactingAdvisor implements CallAdvisor {

    private static final String EMAIL = "[\\w.+-]+@[\\w-]+\\.[\\w.]+";

    @Override
    public ChatClientResponse adviseCall(ChatClientRequest request, CallAdvisorChain chain) {
        Prompt redacted = request.prompt().augmentUserMessage(
                user -> user.mutate().text(user.getText().replaceAll(EMAIL, "[email]")).build());
        return chain.nextCall(request.mutate().prompt(redacted).build());   // pass it on down the chain
    }

    @Override
    public String getName() { return "RedactingAdvisor"; }

    /** Run BEFORE the memory advisor (HIGHEST_PRECEDENCE + 200) so the stored history is redacted too. */
    @Override
    public int getOrder() { return Ordered.HIGHEST_PRECEDENCE + 100; }
}
```

- `CallAdvisor` handles blocking calls; also implement `StreamAdvisor` for `.stream()`. For simple before/after logic, `BaseAdvisor` offers `before(...)` / `after(...)` hooks.
- Requests and prompts are **immutable**: create a modified copy (`mutate()`) and pass it on.

The support client wires three advisors as defaults:

```
@Bean
ChatClient supportClient(ChatClient.Builder builder, ChatMemory chatMemory) {
    return builder
            .defaultSystem("You are a polite support assistant for Acme Shop. Answer briefly.")
            .defaultAdvisors(
                    MessageChatMemoryAdvisor.builder(chatMemory).build(),
                    new RedactingAdvisor(),
                    SimpleLoggerAdvisor.builder()         // logs at DEBUG; here: one short line each way
                            .requestToString(req -> req.prompt().getUserMessage().getText())
                            .responseToString(res -> res.getResult().getOutput().getText())
                            .build())
            .build();
}
```

`SimpleLoggerAdvisor` logs at **DEBUG** (enable `logging.level.org.springframework.ai.chat.client.advisor=DEBUG` in `application.properties`); by default it logs whole request/response objects, shortened above. The test sends `alice@example.com` and a shipping question:

```
model saw:      Please send the invoice to [email]
memory stored:  Please send the invoice to [email]
LOG SimpleLoggerAdvisor    : request: Do you ship to Canada?
LOG SimpleLoggerAdvisor    : response: (fake model) You asked: Do you ship to Canada?
```

Because the redactor runs *before* the memory advisor, neither the model nor the stored history sees the address; reversed, memory would store it and resend it in every later prompt.

Common mistakes

- Wrong order: memory before RAG and logging if they should see history; redaction before anything that stores or sends text.
- Forgetting `chain.nextCall(...)` — the model is never called.
- Logging full prompts in production (personal data, size). Spring AI's observations keep prompt/completion content **out** of logs and traces by default (`spring.ai.chat.client.observations.log-prompt=false`).

In the interview

Q: What is an advisor in Spring AI? → An interceptor around every `ChatClient` call, ordered by `getOrder()`, that can modify the request (add memory, retrieved documents, redaction) or the response. Memory, RAG, logging and — since 2.0 — the tool-calling loop are all advisors.

- *Like what else in Spring?* → A servlet filter chain or AOP `@Around` advice: each link calls the next.
- *Where to enforce "no personal data to the provider"?* → An early advisor, so it covers every call and stored memory.

17.6 Chat memory

What it is

LLM APIs are **stateless**: each request must carry the whole conversation, so "memory" means your app stores past messages and **re-sends** the relevant ones. Spring AI splits this into:

Piece	Role	Default
`ChatMemoryRepository`	stores messages per conversation id	`InMemoryChatMemoryRepository`; JDBC (`spring-ai-starter-model-chat-memory-repository-jdbc`), Cassandra, Neo4j, Redis, ...
`ChatMemory`	decides **which** messages to keep	`MessageWindowChatMemory`: the last **20** messages (system messages are kept)
memory advisor	loads history into the prompt, saves the new exchange	`MessageChatMemoryAdvisor` (history as messages) or `VectorStoreChatMemoryAdvisor` (relevant old messages from a vector store)

Why a window? The context window is limited and every token costs (AI 6.9); long conversations need trimming or summarising.

How it works

1. Every call passes a **conversation id** (a user id, a chat session id).
2. The advisor loads that conversation's messages and puts them before the new user message.
3. After the reply, it saves the user message and the assistant reply.

In Spring AI **2.0 the conversation id is required at call time** (the old default id and the builder's `.conversationId(...)` were removed):

```
private String ask(String conversationId, String text) {
    return support.prompt()
            .user(text)
            .advisors(a -> a.param(ChatMemory.CONVERSATION_ID, conversationId))
            .call()
            .content();
}

ask("alice-42", "Hi, my name is Alice.");
ask("alice-42", "What is my name?");
ask("bob-7",    "What is my name?");
```

Output (`AdvisorTests`):

```
alice> (fake model) You asked: Hi, my name is Alice.
alice> Your name is Alice.
bob>   I don't know your name.
second call for alice-42 sent these messages to the model:
  SYSTEM: You are a polite support assistant for Acme Shop. Answer briefly.
  USER: Hi, my name is Alice.
  ASSISTANT: (fake model) You asked: Hi, my name is Alice.
  USER: What is my name?
```

The fake has no memory: it "knows" Alice's name only because the advisor re-sent the first exchange. Bob's conversation is separate; the test asserts 4 stored messages for `alice-42`, 2 for `bob-7`.

Common mistakes

- One fixed conversation id for everybody — users see each other's history. Derive it from the authenticated user and session (Chapter 10).
- In-memory repository in a cluster (e.g. three instances behind a load balancer): different memory per instance, lost on restart. Use JDBC or another shared repository.
- Unbounded history: cost and latency grow every turn. Keep a window; summarise older turns.

17.7 Structured output

What it is

Structured output returns the answer as data (record, list, map). A **converter** (1) appends format instructions with a **JSON Schema** (a JSON document describing required fields and types) of your type to the prompt and (2) parses the reply with Jackson 3. Why schemas help: AI 6.4.

```
public record MovieRecommendation(String title, int year, List<String> genres, double score) {}

MovieRecommendation movie = builder.build().prompt()
        .user("Recommend one science-fiction movie about survival.")
        .call()
        .entity(MovieRecommendation.class);          // schema added, reply parsed
```

Output (`StructuredOutputTests`):

```
user message sent to the model (first 3 lines):
  | Recommend one science-fiction movie about survival.
  | Your response should be in JSON format.
  | Do not include any explanations, only provide a RFC8259 compliant JSON response following this format without
deviation.
  | ... (31 lines in total, including a JSON Schema)
parsed record: MovieRecommendation[title=The Martian, year=2015, genres=[sci-fi, survival], score=8.0]
```

The converter added ~30 lines to the prompt (tokens you pay for), and the fake replied with JSON **inside a Markdown code fence** (```` ```json ````), as models often do; the converter strips the fence before parsing.

The converter also works standalone:

```
var converter = new BeanOutputConverter<>(MovieRecommendation.class);
MovieRecommendation m = converter.convert("""
        {"title":"Arrival","year":2016,"genres":["sci-fi","drama"],"score":7.9}""");
```

```
converted: MovieRecommendation[title=Arrival, year=2016, genres=[sci-fi, drama], score=7.9]
schema mentions the fields: true, true
```

Options

Option	How	Notes
a class / record	`.entity(MovieRecommendation.class)`	most common
generic types	`.entity(new ParameterizedTypeReference<List<MovieRecommendation>>() {})`	lists, maps
map / list converters	`MapOutputConverter`, `ListOutputConverter`	loose structures
native structured output	`.advisors(AdvisorParams.ENABLE_NATIVE_STRUCTURED_OUTPUT)`	the provider itself constrains output to the schema (only supporting providers/models, e.g. OpenAI; check your provider's page) — more reliable than instructions
validation + retry	`StructuredOutputValidationAdvisor` (2.0)	checks the JSON against the schema; asks the model again on mismatch

Common mistakes

- Trusting the parsed object blindly — right *shape*, possibly wrong *values*. Validate business rules (Bean Validation, 5.6).

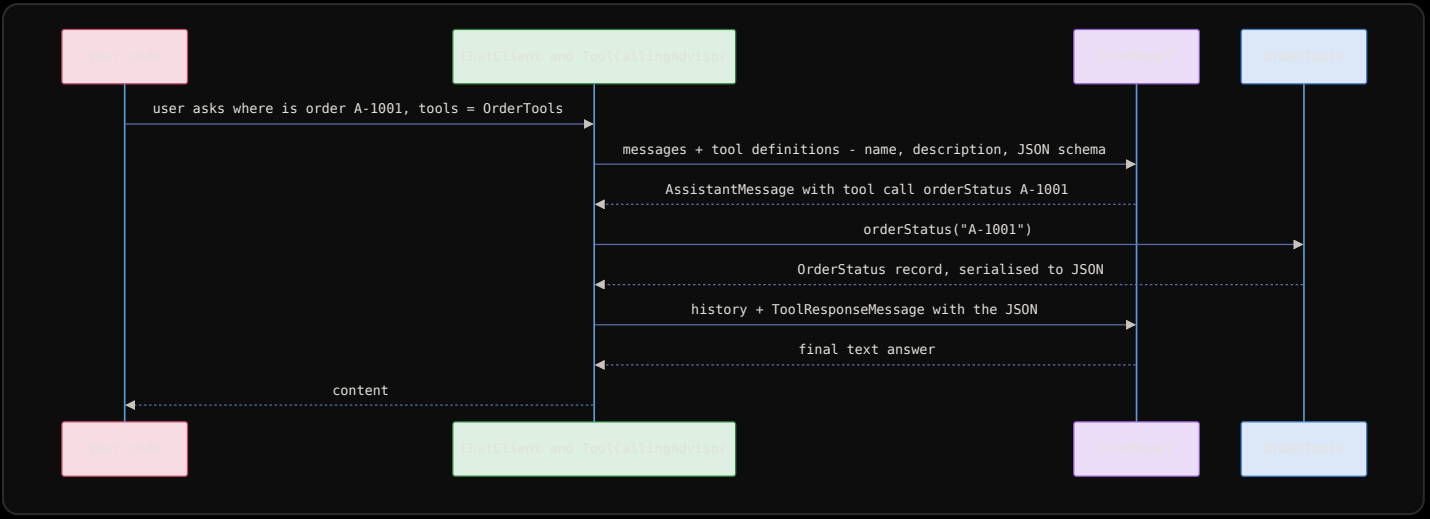
- Huge nested types — the schema costs tokens; keep output types small.
- Forgetting parsing can fail — catch, retry once, or use the validation advisor.

17.8 Tool calling

What it is

Tool calling (function calling) lets the model ask your application to run a function — look up an order, check stock, call an API — and answer with the result. The model **never runs code**: it returns a request like "call `orderStatus` with `{"orderId":"A-1001"}`", and your app decides whether to run it. Concept: AI 6.5, AI 9.3.

How it works



1. You mark methods with `@Tool` and pass the object with `.tools(...)` (or `.defaultTools(...)` on the builder).
2. Spring AI generates a **tool definition** from the method: name, description, and a JSON Schema of the parameters.
3. The model replies with a **tool call** instead of text.
4. The `ToolCallingAdvisor` (auto-registered since 2.0) runs the method, converts the result to JSON, adds a `ToolResponseMessage`, and calls the model **again**.
5. The loop repeats until the model answers with text.

```
public class OrderTools {

    public record OrderStatus(String orderId, String status, String eta) {}

    @Tool(name = "orderStatus", description = "Look up the delivery status of an order by its id, e.g. A-1001")
    public OrderStatus orderStatus(@ToolParam(description = "the order id") String orderId) {
        return new OrderStatus(orderId, "SHIPPED", "2026-10-02");   // a real app would call the order service
    }
}

String answer = builder.build().prompt()
        .user("Where is my order A-1001?")
        .tools(new OrderTools())              // expose the @Tool methods for this request
        .call()
        .content();
```

Output (`ToolCallingTests`) — the test inspects the two prompts the model received:

```
model calls: 2
1) tools offered to the model: [orderStatus]
2) model asked for: orderStatus {"orderId":"A-1001"}
3) Spring ran the method, sent back: {"orderId":"A-1001","status":"SHIPPED","eta":"2026-10-02"}
4) final answer: Here is what I found via orderStatus: {"orderId":"A-1001","status":"SHIPPED","eta":"2026-10-02"}
no tools -> (fake model) You asked: Where is my order A-1001?
```

One user question cost **two model calls** — every tool round trip adds latency and tokens.

Details worth knowing (Spring AI 2.0)

Topic	Fact
description	the model decides *when* to call a tool from its **description** — write it like documentation for a new colleague
`returnDirect = true`	return the tool result straight to the caller without a second model call
`ToolContext`	pass hidden data (tenant id, user id) to tools with `.toolContext(Map)`; it is **not** sent to the model
limits	at most **40 calls per tool** and **150 in total** per request by default (`spring.ai.tools.limits.*`) against runaway loops
resolution	only tools attached to the request or client can run; resolving arbitrary beans by name is **off** by default in 2.0
errors	by default a tool exception is sent back to the model as an error message (`spring.ai.tools.throw-exception-on-error=false`) so it can recover
programmatic tools	`FunctionToolCallback` / `MethodToolCallback` build tools without annotations

Security: tools are an attack surface

Tools are an **attack surface**: the model's tool arguments are **untrusted input**, and prompt-injection text in a retrieved document or user message can make the model call tools with harmful arguments. Rules:

- Check authorization **inside** the tool for the real user (from `ToolContext` or the security context, never from model-supplied arguments).
- Least power: read-only where possible, human confirmation for anything irreversible (AI 9.4).
- Validate arguments like any request body.

In the interview

Q: Explain tool calling in Spring AI. → You annotate methods with `@Tool`; Spring AI sends their JSON-schema definitions with the prompt. When the model returns a tool call, the `ToolCallingAdvisor` executes the method, sends the result back as a tool message, and calls the model again until it produces a final answer. The model only *requests* calls; the application executes them.

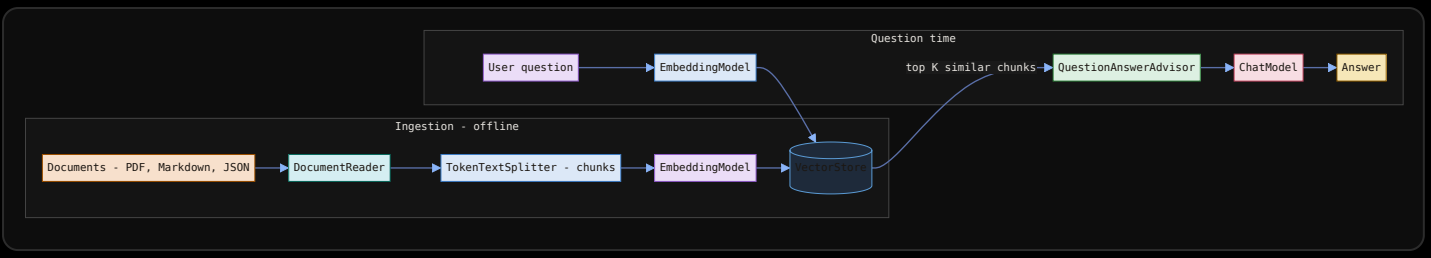
- *Stop an infinite loop?* → Tool-call limits (40 per tool, 150 total by default in 2.0) plus timeouts.
- *Pass the current user to a tool safely?* → `ToolContext` or the security context — never a model-chosen user id argument.

17.9 RAG: retrieval-augmented generation

What it is

RAG answers from **your** documents: find the most relevant chunks, paste them into the prompt, tell the model to answer only from them. It reduces made-up answers and uses private or fresh data without retraining. Design questions (chunking — splitting documents, hybrid search — keyword + meaning,

reranking — re-sorting with a stronger model, access control, evaluation): AI Chapter 7. Here: the Spring AI wiring.



Piece	Spring AI type	Examples
read	`DocumentReader`	`TextReader`, `JsonReader`, `MarkdownDocumentReader`, `PagePdfDocumentReader`, `TikaDocumentReader`
split	`TokenTextSplitter`	chunks of about 800 tokens by default
embed	`EmbeddingModel`	OpenAI `text-embedding-3-small`, Ollama, ONNX transformers (local)
store and search	`VectorStore`	`SimpleVectorStore` (in memory), PgVector, Redis, Elasticsearch, Qdrant, Milvus, ...
augment	advisor	`QuestionAnswerAdvisor` (simple), `RetrievalAugmentationAdvisor` (modular)

The embedding model and the vector store

An **embedding** is a vector representing a text so that similar meanings point in similar directions (AI 7.2); similarity is usually the **cosine** of the angle (1 = same direction, 0 = unrelated).

Our `FakeEmbeddingModel` hashes each word (skipping stop words like "the", cutting words to 5 letters) into one of 256 slots and normalises the vector to length 1, so texts sharing words become similar. A real model captures *meaning* ("refund" ≈ "money back"), which word hashing can't.

```
@Bean
VectorStore vectorStore(EmbeddingModel embeddingModel) {
    SimpleVectorStore store = SimpleVectorStore.builder(embeddingModel).build();
    store.add(List.of(
            new Document("Refunds are issued to the original payment method within 5 business days.",
                    Map.of("source", "refund-policy.md")),
            new Document("Standard shipping takes 3 to 7 days. Express shipping arrives the next day.",
                    Map.of("source", "shipping.md")),
            new Document("You can change your password under Settings, then Security.",
                    Map.of("source", "account.md"))));
    return store;
}
```

`add(...)` embeds each document and stores the vector with its text and **metadata** (source, tenant, date — for filtering and citations).

```
List<Document> hits = vectorStore.similaritySearch(
        SearchRequest.builder().query("How long do refunds take?").topK(2).build());

List<Document> filtered = vectorStore.similaritySearch(SearchRequest.builder()
        .query("How long do refunds take?").topK(2).similarityThreshold(0.1).build());
```

Output (`RagTests`):

```
score 0.192  refund-policy.md   Refunds are issued to the original payment method within 5 business days.
score 0.000  shipping.md        Standard shipping takes 3 to 7 days. Express shipping arrives the next day.
with similarityThreshold(0.1): 1 document(s)
```

`topK(2)` always returns two documents, even an irrelevant one (score 0.000); a **similarity threshold** drops weak matches — without it RAG pastes noise into the prompt. Tune K and the threshold on real questions.

`QuestionAnswerAdvisor`

```
ChatClient chat = builder
        .defaultAdvisors(QuestionAnswerAdvisor.builder(vectorStore)
                .searchRequest(SearchRequest.builder().topK(1).similarityThreshold(0.2).build())
                .build())
        .build();

String answer = chat.prompt().user("How fast is express shipping?").call().content();
```

Output — the user message after the advisor rewrote it:

```
augmented user message:
  | How fast is express shipping?
  |
  | Context information is below, surrounded by ---------------------
  |
  | ---------------------
  | Standard shipping takes 3 to 7 days. Express shipping arrives the next day.
  | ---------------------
  |
  | ...
answer: According to our docs: Standard shipping takes 3 to 7 days. Express shipping arrives the next day.
```

The rest of the default template tells the model to answer from the context and say so when the answer isn't there. Replace it with `.promptTemplate(...)`; restrict the search per request with a metadata **filter expression**, e.g. `a.param(QuestionAnswerAdvisor.FILTER_EXPRESSION, "tenant == 'acme'")` — how document-level permissions are enforced (AI 7.9). Retrieved documents are stored in the response context under `QuestionAnswerAdvisor.RETRIEVED_DOCUMENTS`, for citations.

For more control, `RetrievalAugmentationAdvisor` (module `spring-ai-rag`) builds a pipeline from parts: `RewriteQueryTransformer` / `CompressionQueryTransformer` (improve the query), `MultiQueryExpander`, `VectorStoreDocumentRetriever`, a `DocumentJoiner`, post-processors (e.g. reranking) and `ContextualQueryAugmenter`.

A production vector store (needs a database — not run)

`SimpleVectorStore` keeps everything in memory (`save` / `load` to a JSON file) — fine for tests and demos, not real data. With PostgreSQL + `pgvector`:

```
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-starter-vector-store-pgvector</artifactId>
</dependency>
```

```
# (needs PostgreSQL with pgvector — not run)
spring.datasource.url=jdbc:postgresql://localhost:5432/docs
spring.ai.vectorstore.pgvector.initialize-schema=true
spring.ai.vectorstore.pgvector.index-type=hnsw
spring.ai.vectorstore.pgvector.distance-type=cosine-distance
spring.ai.vectorstore.pgvector.dimensions=1536
```

`dimensions` must match the embedding model (1536 for `text-embedding-3-small`); changing the model means **re-embedding every document**. `hnsw` (Hierarchical Navigable Small World) is an approximate nearest-neighbour index (AI 7.3); `initialize-schema=true` suits development, but production creates the table with Flyway (8.16).

Common mistakes

- No similarity threshold — irrelevant chunks confuse the model.
- Chunks too large (costly, diluted) or too small (no context) — start around a few hundred tokens with some overlap.
- Ignoring permissions — store tenant/role in metadata and filter.
- Mixing embeddings from two models in one store — vectors aren't comparable.

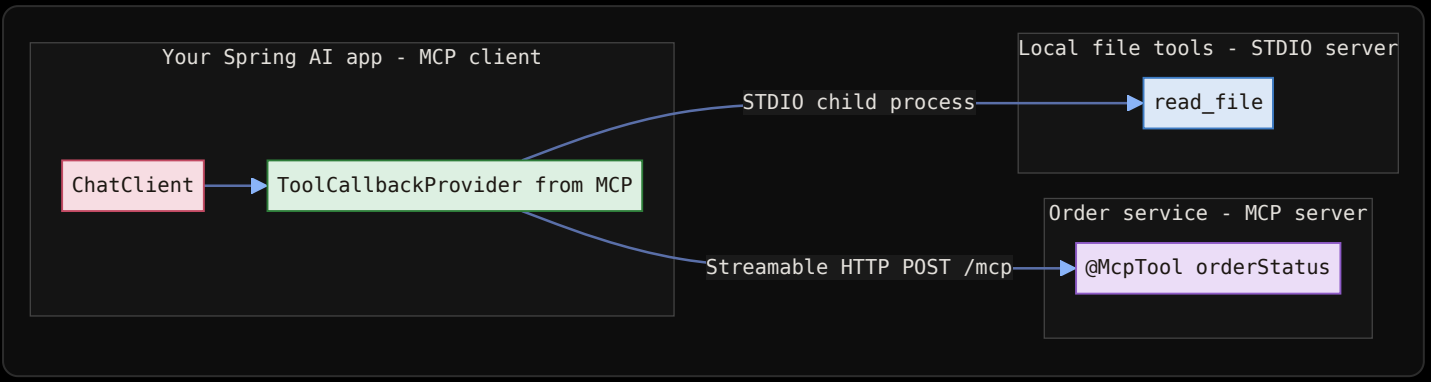
17.10 MCP: the Model Context Protocol

What it is

MCP is an open protocol (started by Anthropic in November 2024, now widely adopted) standardising how an AI application discovers and calls **tools**, reads **resources** and uses **prompts** exposed by another process — "USB for AI tools": write a tool server once, and any MCP host (Claude Desktop, IDE assistants, your Spring AI app) can use it. Concepts and security: AI 6.10, AI 9.8.

Role	What it does	Spring AI starter
MCP server	exposes tools/resources/prompts	`spring-ai-starter-mcp-server` (STDIO), `spring-ai-starter-mcp-server-webmvc`, `spring-ai-starter-mcp-server-webflux`
MCP client	connects to servers, turns their tools into `ToolCallback`s	`spring-ai-starter-mcp-client`, `spring-ai-starter-mcp-client-webflux`

Transports: **STDIO** (the host starts the server as a child process and they talk over its input/output streams — local tools) and **Streamable HTTP** (remote; the default for Spring AI 2.0's web servers, endpoint `POST /mcp`). The older SSE transport is deprecated since 2.0.



An MCP server (snippet — not run)

```java
// spring-ai-starter-mcp-server-webmvc on the classpath
import org.springframework.ai.mcp.annotation.McpTool;
import org.springframework.ai.mcp.annotation.McpToolParam;
import org.springframework.stereotype.Component;

@Component
public class OrderMcpTools {

    @McpTool(name = "orderStatus", description = "Delivery status of an order by id")
    public OrderStatus orderStatus(@McpToolParam(description = "order id", required = true) String orderId) {
        return orderService.status(orderId);
    }
}
```

```properties
spring.ai.mcp.server.name=order-tools
# already the default in 2.0 (endpoint /mcp); shown for clarity
spring.ai.mcp.server.protocol=STREAMABLE
```

In Spring AI 2.0 the MCP annotations live in `org.springframework.ai.mcp.annotation` (from the `org.springaicommunity.mcp` community project in 1.1); beans with `@McpTool`, `@McpResource`, `@McpPrompt` methods are discovered automatically.

An MCP client (snippet — needs a running server, not run)

```
spring.ai.mcp.client.streamable-http.connections.orders.url=http://localhost:8081
# a local STDIO server, started by the client:
spring.ai.mcp.client.stdio.connections.files.command=npx
spring.ai.mcp.client.stdio.connections.files.args=-y,@modelcontextprotocol/server-filesystem,/data/docs
```

```
@Bean
ChatClient agent(ChatClient.Builder builder, ToolCallbackProvider mcpTools) {
    return builder.defaultToolCallbacks(mcpTools).build();   // MCP tools behave like @Tool methods
}
```

The client auto-configuration exposes remote tools as a `ToolCallbackProvider` bean (`spring.ai.mcp.client.toolcallback.enabled=true` by default), so §17.8's tool loop works unchanged.

Trade-offs

	In-process `@Tool`	MCP tool
latency	method call	network or process hop
reuse	this app only	any MCP host, any language
deployment	ships with the app	separate service, separate lifecycle
security	your normal code review	you now trust a remote server's tool descriptions and results — authenticate it (OAuth 2 for HTTP servers), allow-list tools, treat results as untrusted input

`@Tool` for logic belonging to your app; MCP to share tools across teams or AI hosts, or to consume existing servers.

17.11 Production concerns

Concern	What Spring AI gives you	Also see
observability	Micrometer observations: `spring.ai.chat.client`, `spring.ai.advisor`, `gen_ai.client.operation` (model call, with `gen_ai.client.token.usage` metrics), `spring.ai.tool`, `db.vector.client.operation`. Prompt/completion content is **not** recorded unless you enable `log-prompt` / `log-completion`	Chapter 15
cost	token usage in `chatResponse().getMetadata().getUsage()`	AI 6.6
retries and timeouts	provider SDK retries (e.g. `spring.ai.openai.chat.max-retries`, default 3) and `spring.ai.retry.*` for providers using Spring's retry; set HTTP timeouts	AI 6.7, 4.7
streaming	`.stream().content()` returns `Flux<String>`; send it to browsers with SSE	Chapter 13
blocking calls	model calls take seconds; on Spring MVC enable virtual threads (`spring.threads.virtual.enabled=true`) so waiting is cheap	Java 15.4
safety	redaction advisor, `SafeGuardAdvisor`, output validation, tool permissions	AI Chapter 12
testing	fake `ChatModel` for logic, evaluators for quality	§17.2

Common mistakes (Spring AI, whole part)

- Calling the model inside a database transaction (Chapter 7) — the transaction and connection stay open for seconds.

- No timeout or limit on tool loops and output tokens.
- Model text reaching SQL, shell commands or HTML without validation or escaping.
- Hard-coding a model id the provider later retires — keep it in configuration.
- Tests calling the real API on every build.

In the interview

Q: How would you add an AI assistant to an existing Spring Boot service? → Spring AI BOM + a model starter, key and model in properties, a `ChatClient` bean with a system prompt and advisors (memory with a per-user conversation id, RAG over our documents with metadata filters, logging/redaction), business functions as `@Tool` methods that check authorization themselves, typed results with `.entity(...)`. Test logic with a fake `ChatModel`, measure quality with an evaluation set, watch token usage and latency through Micrometer.

- *Why Spring AI rather than the provider's SDK?* → Portability across providers plus Spring integration: auto-configuration, observability, advisors, vector stores, testing. Provider-specific features remain reachable via native options.
- *Keeping answers grounded (based only on real sources)?* → RAG with a similarity threshold, an "answer only from the context" template, citations from retrieved-document metadata, faithfulness evaluation.

Spring AI TL;DR

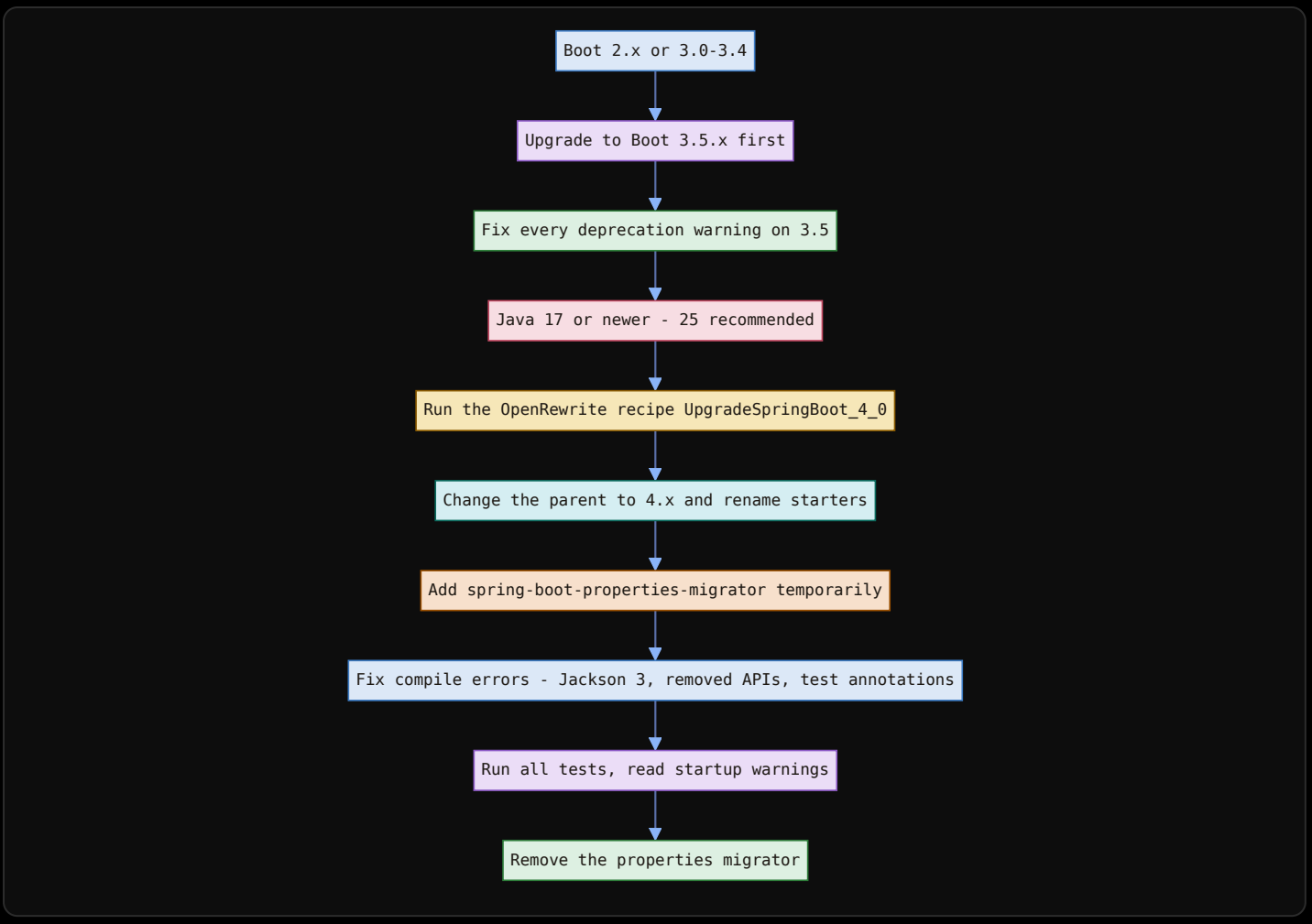
- Spring AI **2.0.x** for Boot 4 (1.1.x for Boot 3.5). Import `spring-ai-bom`, add one model starter, set `spring.ai.<provider>.api-key` and `...chat.model`.
- `ChatClient` = fluent API over `ChatModel`; `.content()`, `.entity(...)`, `.stream()`.
- **Advisors** wrap every call (ordered, lower first): logging, memory, RAG, tool calling, your own (redaction).
- **Memory** re-sends history; pass `ChatMemory.CONVERSATION_ID` per call; window of 20 by default.
- **Structured output**: schema instructions + Jackson parsing; native mode and validation advisor for reliability.
- **Tools**: `@Tool` methods; the model requests, `ToolCallingAdvisor` executes and loops (limits 40/150). Authorize inside tools.
- **RAG**: reader → splitter → `EmbeddingModel` → `VectorStore`; `QuestionAnswerAdvisor` with threshold and metadata filters.
- **MCP**: share tools across processes; `@McpTool` servers, client tools arrive as a `ToolCallbackProvider`.
- Test with a **fake model**; evaluate quality separately.

17.12 What's new: how to read the rest of this chapter

The Spring generation of **November 2025** (Framework 7.0 on 13 Nov, Boot 4.0 on 20 Nov, Security 7.0) is the biggest change since Boot 3 moved `javax.*` to `jakarta.*` (Java's enterprise APIs were handed to the Eclipse Foundation as **Jakarta EE** and renamed). This book targets **Boot 4.1.1**, whose BOM gives Framework 7.0.9, Security 7.1.1, Hibernate ORM 7.4, Spring Data 2026.0, Jackson 3.1.5, Tomcat 11.0.24 and JUnit 6.0.3 (1.4).

The tables below are a **map**, one line per change with a pointer to the chapter that teaches it — checked against the Boot 4.0 migration guide, Boot 4.0/4.1 release notes, Framework 7.0 release notes and the Spring Security 7 reference.

A safe upgrade path (Boot 3.x → 4.x)



1. **Go to 3.5 first.** The migration guide assumes 3.5; code deprecated in 3.x is removed in 4.0.
2. **Automate the boring part** with OpenRewrite (rule-based automated source changes called *recipes*; not run here):

   ```
   mvn -U org.openrewrite.maven:rewrite-maven-plugin:run \
     -Drewrite.recipeArtifactCoordinates=org.openrewrite.recipe:rewrite-spring:RELEASE \
     -Drewrite.activeRecipes=org.openrewrite.java.spring.boot4.UpgradeSpringBoot_4_0
   ```

   It updates versions, renames starters and properties, and applies the Framework 7 / Security 7 sub-recipes. Review its diff like any change.
3. `spring-boot-properties-migrator` (temporary runtime dependency) reports renamed/removed properties at startup and maps old names while you fix them.
4. Stuck? The **"classic" starters** (`spring-boot-starter-classic`, `spring-boot-starter-test-classic`) restore the old all-in-one auto-configuration layout to get you compiling; then move to modular starters.

17.13 Spring Boot 3.x → 4.x migration table

Baselines

Item	Boot 3.x	Boot 4.x	Where in this book
Java	17+	17+ (25 recommended; this book uses 25)	1.4
Spring Framework	6.x	7.0	§17.14
Jakarta EE	10 (Servlet 6.0 — the Java web request/response API)	**11** (Servlet 6.1, JPA 3.2, Bean Validation 3.1)	1.4
Embedded servers	Tomcat 10.1, Jetty 12, Undertow	Tomcat 11, Jetty 12.1; **Undertow removed** (no Servlet 6.1 support)	Chapter 1
Hibernate ORM	6.x	7.x (7.4 in Boot 4.1)	Chapter 8
JUnit	5	**6** (JUnit 4 support in the TestContext framework deprecated)	Chapter 12
Kotlin / GraalVM	1.7+ / 22.3+	Kotlin 2.2+ / GraalVM 25+	Chapter 15
Gradle	7.x / 8.x	8.14+ or 9	1.6

Modularization and starters

Change	What to do	Where
`spring-boot-autoconfigure` split into **one module per technology** (`spring-boot-webmvc`, `spring-boot-jdbc`, `spring-boot-jackson`, ...) with its own starter and **test starter**	use the technology starters; imports of auto-configuration classes (for `exclude`) changed packages	3.14
`spring-boot-starter-web` → **`spring-boot-starter-webmvc`**	rename (the old name remains as a deprecated alias)	1.7
`spring-boot-starter-aop` → **`spring-boot-starter-aspectj`**	rename	4.2
`spring-boot-starter-oauth2-client` / `-resource-server` / `-authorization-server` → `spring-boot-starter-security-oauth2-...`	rename	Chapter 11
`spring-boot-starter-web-services` → `spring-boot-starter-webservices`	rename	—
test support per module: `spring-boot-starter-webmvc-test`, `-data-jpa-test`, `-restclient-test`, ...	add the test starter for each technology you test	Chapter 12
Flyway / Liquibase auto-configured only with **`spring-boot-starter-flyway`** / `-liquibase` (the library alone is no longer enough)	add the starter	8.16
`RestClient` / `RestTemplate` builders come from **`spring-boot-starter-restclient`**	add it for client code	Chapter 6
`@EntityScan` moved to `org.springframework.boot.persistence.autoconfigure`	fix the import	Chapter 8
`hibernate-jpamodelgen` → `hibernate-processor`	rename the annotation processor (compiler plug-in generating code from annotations)	Chapter 8
Spring Batch runs **without a database** by default	use `spring-boot-starter-batch-jdbc` for JDBC job metadata	—

Jackson 3

Change	What to do	Where
Jackson 3 is the default; packages and group ids moved from `com.fasterxml.jackson` to **`tools.jackson`** (annotations such as `@JsonProperty` keep `com.fasterxml.jackson.annotation`)	fix imports; `ObjectMapper` usage mostly becomes `JsonMapper`	5.5
`Jackson2ObjectMapperBuilderCustomizer` → `JsonMapperBuilderCustomizer`; `@JsonComponent` → `@JacksonComponent`; `@JsonMixin` → `@JacksonMixin`	rename	5.5
JSON-specific parser/generator features moved from `spring.jackson.parser.*` / `generator.*` to **`spring.jackson.json.read.*`** / **`spring.jackson.json.write.*`** (`spring.jackson.read.*` / `write.*` remain for format-neutral stream features); `spring.jackson.use-jackson2-defaults=true` restores Jackson 2 behaviour	update properties	5.5
Jackson 2 still possible through the **deprecated** `spring-boot-jackson2` module (`spring.jackson2.*`); Framework plans to disable its auto-detection in 7.1 and remove it in 7.2	migrate now	5.5
`HttpMessageConverters` bean type deprecated	use `ServerHttpMessageConvertersCustomizer` / `ClientHttpMessageConvertersCustomizer`	Chapter 5

Null-safety, testing, and other code changes

Change	What to do	Where
Boot and Framework annotated with **JSpecify** (`@NullMarked`, `@Nullable`); Spring's own `org.springframework.lang.Nullable` deprecated	use `org.jspecify.annotations` in your code; Kotlin and null checkers may flag new warnings	2.13
`@MockBean` / `@SpyBean` **removed**	`@MockitoBean` / `@MockitoSpyBean`	Chapter 12
`@SpringBootTest` no longer sets up MockMvc, `WebTestClient` or `TestRestTemplate` on its own	add `@AutoConfigureMockMvc`, `@AutoConfigureRestTestClient`, `@AutoConfigureTestRestTemplate`	Chapter 12
`TestRestTemplate` moved to `org.springframework.boot.resttestclient` (module `spring-boot-resttestclient`)	fix imports, or switch to the new **`RestTestClient`**	1.12
`EnvironmentPostProcessor` moved to `org.springframework.boot`; `BootstrapRegistry` to `org.springframework.boot.bootstrap`	update imports and `spring.factories`	Chapter 3
Spring Retry no longer version-managed	declare its version, or move to Framework 7 `@Retryable`	4.7
Spring Authorization Server moved into Spring Security 7 (`spring-authorization-server.version` property gone)	use `spring-security.version`	11.14

Property renames and changed defaults (the most common)

Before (3.x)	After (4.x)
`spring.jackson.parser.*`, `spring.jackson.generator.*`	`spring.jackson.json.read.*`, `spring.jackson.json.write.*`
`spring.data.mongodb.host` / `.port` / `.uri` / `.database` / ...	`spring.mongodb.host` / `.port` / `.uri` / `.database` / ...
`management.health.mongo.enabled`	`management.health.mongodb.enabled`
`server.error.*` (`include-stacktrace`, `include-message`, `path`, `whitelabel.enabled`, ...)	`spring.web.error.*` — the old keys are no longer bound since 4.0.0
`spring.session.redis.*`	`spring.session.data.redis.*` (Spring Session MongoDB left Boot, see below)
`spring.dao.exceptiontranslation.enabled`	`spring.persistence.exceptiontranslation.enabled`
`spring.kafka.retry.topic.backoff.random`	`spring.kafka.retry.topic.backoff.jitter`
DevTools LiveReload on by default	`spring.devtools.livereload.enabled` defaults to **false** (and LiveReload is deprecated in 4.1)
Kubernetes liveness/readiness probes only on Kubernetes	health **probes enabled by default** (`management.endpoint.health.probes.enabled`) — Chapter 15
`server.forward-headers-strategy` applied to war deployments	no longer applies to wars — register a `ForwardedHeaderFilter` yourself

Removed features

Removed in Boot 4.0	Replacement
Undertow	Tomcat or Jetty
executable "fully executable" jar launch scripts	run with `java -jar`, systemd, containers
Maven `CLASSIC` loader implementation	the default loader
optional dependencies packed into the uber (fat) jar	`<includeOptional>true</includeOptional>` if you really need them
Spock integration (Groovy 5 incompatibility)	**back in Boot 4.1** with Spock 2.4
Spring Session Hazelcast / MongoDB (moved to those vendors)	the vendors' own modules
Pulsar reactive support	Spring Pulsar imperative API

New in Boot 4.0

Feature	Where
API versioning auto-configuration (`spring.mvc.apiversion.*`, `spring.webflux.apiversion.*`)	5.9
HTTP service clients auto-configured: `@ImportHttpServices` + `spring.http.serviceclient.<group>.*`	6.6
`spring.http.clients.*` for the auto-configured HTTP client library and timeouts	6.4
`RestTestClient` in tests	1.12, Chapter 12
`spring-boot-starter-opentelemetry` (OTLP metrics and traces)	Chapter 15
`JmsClient` auto-configuration	Chapter 14
several `TaskDecorator` beans combined automatically	Chapter 14
`logging.console.enabled=false` to turn console logging off	Chapter 15
Kotlin serialization starter	—

17.14 Spring Framework 6 → 7

Area	What changed in 7.0	Where
baselines	Java 17 (25 recommended), Jakarta EE 11, Servlet 6.1, JPA 3.2, Bean Validation 3.1, Kotlin 2.2, GraalVM 25, Netty 4.2, JUnit 6	1.4
null-safety	whole codebase annotated with **JSpecify**, including generics, arrays and varargs; JSR 305-style Spring annotations deprecated	2.13
programmatic beans	new `BeanRegistrar` contract for registering beans with code (loops, conditions)	2.14
resilience	`RetryTemplate` / `RetryPolicy` in `spring-core` (`org.springframework.core.retry`), `@Retryable` and `@ConcurrencyLimit` in `spring-context`, enabled with **`@EnableResilientMethods`**; `@Retryable` also adapts to reactive return types	4.7
proxies	CGLIB (subclass-generating proxies) default applied consistently to all proxy processors (including `@Async`); **`@Proxyable`** overrides per bean	4.4
API versioning	first-class in Spring MVC and WebFlux: resolve the version from header, query parameter, path segment or media type; deprecation hints; client-side support in `RestClient`, `WebClient`, HTTP interfaces; testing support	5.9, 6.7
HTTP interface clients	group-based configuration with `@ImportHttpServices` and `HttpServiceGroupConfigurer`; `InputStream` / `OutputStream` support	6.6
`RestTemplate`	deprecated in the reference docs in 7.0, `@Deprecated` planned for 7.1, removal in 8.0	6.1
Jackson 3	default JSON library across MVC, WebFlux, messaging; Jackson 2 support deprecated	5.5
message converters	new `HttpMessageConverters` builder API (`configureMessageConverters(ServerBuilder)`)	Chapter 5
path matching	`PathMatcher` / `AntPathMatcher` for MVC mappings deprecated in favour of `PathPattern`, which now supports `/**` at the start of a pattern; suffix-pattern and trailing-slash matching options removed	5.3
`HttpHeaders`	no longer a `MultiValueMap`; some map methods removed	Chapter 5
data access	JPA 3.2: inject `EntityManagerFactory` / `EntityManager` with `@Autowired`; Hibernate 7 support moved to `orm.jpa.hibernate`; `JdbcClient` gains fetch size, max rows, query timeout	Chapter 7, Chapter 8
messaging	new **`JmsClient`** (like `JdbcClient` / `RestClient` for JMS)	Chapter 14
testing	**`RestTestClient`** (non-reactive `WebTestClient`), test application contexts **paused** when not in use, `@MockitoBean` on non-singleton beans, better `@Nested` injection	Chapter 12
SpEL (Spring Expression Language, `#{...}`)	`Optional` support with `?.` and Elvis (`?:`, default if null); since 7.0.8 a default limit of 10,000 operations per evaluation	—
Java 24+	bytecode metadata read with the JDK **Class-File API** (JEP 484) — transparent	Java Ch. 17
removed	`javax.annotation` / `javax.inject` support (use `jakarta.*`), `spring-jcl` (Apache Commons Logging 1.3 instead), `ListenableFuture` (use `CompletableFuture`), OkHttp3 support, `Theme` support, `webjars-locator-core` (use `webjars-locator-lite`), Undertow-specific classes	2.15

Area	What changed in 7.0	Where
deprecated	JUnit 4 support (`SpringRunner`, ...), `<mvc:*>` XML namespace, document/feed views (PDF, XLS, RSS), BeanShell	—

In the interview

Q: What's new in Spring Framework 7 / Boot 4? → Jakarta EE 11 and Java 17 baseline (25 recommended); JSpecify null-safety; built-in API versioning; HTTP service client groups with Boot auto-configuration; core resilience (`@Retryable`, `@ConcurrencyLimit`, `RetryTemplate`); `BeanRegistrar`; Jackson 3; `RestTestClient`; and in Boot, modular auto-configuration with a starter and test starter per technology.

- *What breaks first on upgrade?* → Jackson imports (`tools.jackson`), renamed starters (web → webmvc, aop → aspectj), Flyway needing its starter (migrations silently stop), removed `@MockBean`, `@SpringBootTest` without `@AutoConfigureMockMvc` (no `MockMvc`), `javax.*` annotations, `.and()`-style security configuration.
- *Still need Spring Retry?* → Usually not: Framework 7's `@Retryable` covers retries with backoff and jitter; circuit breakers → Resilience4j (Chapter 16).

17.15 Spring Security 6 → 7

Change	Details	Where
lambda DSL only	`.and()` chaining removed; one lambda per feature	10.3
`authorizeRequests()` removed	use `authorizeHttpRequests()` (built on `AuthorizationManager`)	10.3
`AntPathRequestMatcher`, `MvcRequestMatcher` removed	**`PathPatternRequestMatcher`** (same syntax as Spring MVC)	10.3
`AccessDecisionManager` / voters	moved out of the main jar to the legacy `spring-security-access` module; use `AuthorizationManager`	10.6
multi-factor authentication	every login mechanism adds a factor authority (`FACTOR_PASSWORD`, `FACTOR_OTT`, `FACTOR_WEBAUTHN`); `@EnableMultiFactorAuthentication` or `AuthorizationManagerFactories.multiFactor()` require several factors	10.4
CSRF for single-page apps	`csrf(c -> c.spa())`	10.8
password encoders	Password4j-based encoders (Argon2, bcrypt, scrypt, PBKDF2, balloon hashing)	10.5
Spring Authorization Server	now shipped inside Spring Security 7	11.14
OAuth2 password grant	client support removed	11.2
PKCE	used by default for confidential clients too	11.9
JWT	`NimbusJwtEncoder.withKeyPair(...)` / `withSecretKey(...)` builders; `JwtTypeValidator` checks the `typ` header	11.3, 11.4
HTTP interfaces + OAuth2	`@ClientRegistrationId` on a whole HTTP interface	11.10
7.1 (in Boot 4.1)	`RestClientOpaqueTokenIntrospector`; `charset` parameter in `WWW-Authenticate: Basic`	11.7, 10.1

17.16 What Boot 4.1 adds (June 2026)

Change in 4.1	Notes	Where
everything **deprecated in 4.0 removed**	upgrade 4.0 → 4.1 with zero deprecation warnings first	—
`layertools` jar mode removed	use the `tools` jar mode for layered images	Chapter 15
gRPC (Google's binary RPC protocol) support (server and client, Netty or Servlet over HTTP/2)	new	—
Spock support back (Spock 2.4, Groovy 5)		Chapter 12
`spring.datasource.connection-fetch=lazy`	wraps the pool in `LazyConnectionDataSourceProxy`: no connection taken until the first statement	7.2
`@ConfigurationProperties` records bind `Optional` parameters as `Optional.empty()` instead of `null`		3.6
`spring.security.oauth2.resourceserver.jwt.authorities-claim-expressions`	map authorities from JWT claims with properties	11.5
observability: context propagation into `@Async` methods, `management.opentelemetry.*` properties, Kafka/RabbitMQ observation conventions		Chapter 15
HTTP clients: cookie handling (`spring.http.clients.cookie-handling`), `InetAddressFilter` against SSRF (server-side request forgery: tricking a server into calling internal addresses)		Chapter 6
`@RedisListener` auto-configuration		Chapter 9
dependency upgrades: Hibernate 7.4, Spring Data 2026.0, Micrometer 1.17		Chapter 8
deprecated: DevTools LiveReload, Derby support		—

Common migration mistakes

- Jumping from Boot 2.7 straight to 4.x — go 2.7 → 3.5 (the `javax` → `jakarta` step) → 4.x.
- Ignoring startup warnings — removed auto-configuration shows up as "bean not found", not a compile error.
- Keeping `spring-boot-properties-migrator` in production.
- Upgrading Boot but not separately versioned projects: **Spring Cloud** 2025.1 "Oakwood" (2025.1.2+ for Boot 4.1; Chapter 16), **Spring AI** 2.0, **springdoc-openapi**'s Boot 4 line (5.12).

17.17 In the interview — quick round

Question	30-second answer
What is Spring AI?	A Spring project giving portable APIs (`ChatModel`, `ChatClient`, `EmbeddingModel`, `VectorStore`) and Boot starters for AI providers, plus advisors for memory, RAG and tool calling. (§17.1)
`ChatModel` vs `ChatClient`?	Provider-level interface vs fluent API with templates, advisors, tools, structured output. (§17.3)
How does chat memory work?	The model is stateless; an advisor stores messages per conversation id and re-sends a window of them. (§17.6)
How do you get a Java object from an LLM?	`.entity(MyRecord.class)`: JSON Schema instructions + Jackson parsing; native structured output where supported. (§17.7)
Who executes a tool?	The application. The model only requests the call; `ToolCallingAdvisor` runs it and loops. (§17.8)
How do you test Spring AI code without paying?	A fake `ChatModel` bean with canned answers; real-model evaluations separately. (§17.2)
Main Boot 4 changes?	Modular starters, Jackson 3, JSpecify, Jakarta EE 11, API versioning, HTTP service clients, `@MockitoBean`, Undertow removed. (§17.13)
Main Framework 7 changes?	JSpecify, API versioning, `@ImportHttpServices`, `@Retryable` / `@ConcurrencyLimit`, `BeanRegistrar`, `RestTestClient`, Jackson 3. (§17.14)
Main Security 7 changes?	Lambda DSL only, `PathPatternRequestMatcher`, MFA factors, Authorization Server included, password grant removed. (§17.15)

Chapter TL;DR

- **Spring AI 2.0.x** is the version for Boot 4.x: BOM + one model starter + `spring.ai.<provider>.api-key` / `.chat.model` (e.g. `gpt-5-mini`, `claude-sonnet-5-5`, `claude-opus-5-5`). Set the model explicitly.
- `ChatClient` over `ChatModel`; prompts are messages (system, user, assistant, tool); `PromptTemplate` fills `{placeholders}` and fails on missing values.
- **Advisors** are the extension point: logging, memory (pass `ChatMemory.CONVERSATION_ID`), RAG (`QuestionAnswerAdvisor` + threshold + filters), tool calling (auto-registered in 2.0), your own (redaction). Order matters.
- **Structured output** with `.entity(...)`; **tools** with `@Tool` (authorize inside, limits 40/150); **MCP** to share tools across processes (`@McpTool`, Streamable HTTP).
- Everything above the model runs offline against a **fake** `ChatModel` — 14 tests in this chapter prove it.
- **Boot 3 → 4**: go via 3.5, run OpenRewrite, rename starters (webmvc, aspectj, security-oauth2-*), add test and Flyway starters, move to Jackson 3 (`tools.jackson`), `@MockitoBean`, JSpecify, Jakarta EE 11; Undertow gone.
- **Framework 7**: API versioning, HTTP service groups, core resilience, `BeanRegistrar`, `RestTestClient`, JSpecify; `RestTemplate` on its way out.
- **Security 7**: lambda DSL only, `authorizeHttpRequests`, `PathPatternRequestMatcher`, MFA, Authorization Server inside.

Next: Chapter 18 — Spring interview questions turns the whole book into questions and answers for revision.